Kademlia DHT 구현 및 동작 방식 분석

개요

이 문서는 Rust libp2p의 Kademlia DHT (Distributed Hash Table) 구현체를 상세히 분석합니다. Kademlia는 분산 해시 테이블을 구현하는 P2P 프로토콜로, 피어 검색, 콘텐츠 라우팅, 그리고 키-값 저장을 제공합니다.

1. Kademlia DHT 기본 개념

1.1 핵심 매개변수

```rust
/// K_VALUE: Kademlia 사양의 'k' 매개변수
/// - 버킷의 최대 노드 수
/// - 기본 복제 인수
/// - 현재 값: 20
pub const K_VALUE: NonZeroUsize = NonZeroUsize::new(20).unwrap();

/// ALPHA_VALUE: Kademlia 사양의 'α' 매개변수
/// - 반복 쿼리의 병렬성 수준
/// - 현재 값: 3
pub const ALPHA_VALUE: NonZeroUsize = NonZeroUsize::new(3).unwrap();
```

1.2 거리 계산

Kademlia는 XOR 거리 메트릭을 사용합니다:

- 두 키 간의 거리는 XOR 연산으로 계산
- 거리가 가까울수록 라우팅 테이블에서 우선순위가 높음
- 256비트 키 공간을 사용하여 최대 256개의 버킷 지원

2. 핵심 구성 요소

2.1 Behaviour 구조체

```
pub struct Behaviour<TStore> {
/// Kademlia 라우팅 테이블
kbuckets: KBucketsTable<kbucket::Key<PeerId>, Addresses>,
/// 현재 활성 쿼리들
queries: QueryPool,
/// 연결된 피어들
connected_peers: FnvHashSet<PeerId>,
/// 레코드 저장소
store: TStore,
/// 기타 설정 및 상태 정보
// ...
}
```

2.2 라우팅 테이블 (KBucketsTable)

```rust
pub(crate) struct KBucketsTable<TKey, TVal> {
/// 로컬 피어를 식별하는 키
local_key: TKey,
/// 라우팅 테이블을 구성하는 버킷들
buckets: Vec<KBucket<TKey, TVal>>,
/// 버킷당 최대 노드 수
bucket_size: usize,
/// 적용된 대기 항목들
applied_pending: VecDeque<AppliedPending<TKey, TVal>>,
}
```

버킷 구조

- **고정 크기 배열**: 256개의 버킷 (0-255)
- **거리 기반 분할**: 각 버킷은 특정 거리 범위를 담당
- **LRU 정책**: 가장 최근에 연결되지 않은 노드부터 제거
- **대기 항목**: 버킷이 가득 찬 경우 대기 상태로 관리

2.3 쿼리 시스템 (QueryPool)

```rust
pub(crate) struct QueryPool {
next_id: usize,
config: QueryConfig,
queries: FnvHashMap<QueryId, Query>,
```

```
}

pub(crate) struct Query {
id: QueryId,
peers: QueryPeers,
stats: QueryStats,
info: QueryInfo,
pending_rpcs: SmallVec<[(PeerId, HandlerIn); K_VALUE.get()]>,
}
```

쿼리 유형

1. **Fixed Query**: 고정된 피어 집합에 대한 쿼리
2. **Closest Query**: 목표 키에 가장 가까운 피어들을 찾는 반복 쿼리
3. **Disjoint Query**: 보안 강화를 위한 분리된 경로 쿼리

3. 주요 동작 메커니즘

3.1 피어 검색 (Peer Discovery)

```
impl<TStore> Behaviour<TStore> {
/// 주어진 키에 가장 가까운 피어들을 찾는 반복 쿼리 시작
pub fn get_closest_peers<K>(&mut self, key: K) -> QueryId
where
K: Into<kbucket::Key<K>> + Into<Vec<u8>> + Clone,
```

```
{
self.get_closest_peers_inner(key, K_VALUE)
}
}
```

동작 과정:

1. 로컬 라우팅 테이블에서 목표에 가장 가까운 α개 피어 선택
2. 선택된 피어들에게 FIND_NODE 요청 전송
3. 응답으로 받은 더 가까운 피어들로 쿼리 계속
4. k개의 가장 가까운 피어를 찾을 때까지 반복

3.2 레코드 저장 및 검색

레코드 구조

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct Record {
/// 레코드의 키
pub key: Key,
/// 레코드의 값
pub value: Vec<u8>,
/// 원본 게시자
pub publisher: Option<PeerId>,
/// 만료 시간
pub expires: Option<Instant>,
```

```
}
```

PUT 연산

```
pub fn put_record(&mut self, record: Record, quorum: Quorum) -> Result<QueryId,
// 1. 로컬 저장소에 레코드 저장
// 2. 키에 가장 가까운 k개 피어 찾기
// 3. 해당 피어들에게 레코드 복제
// 4. 쿼럼 조건 만족 시 성공 반환
}
```

GET 연산

```
pub fn get_record(&mut self, key: record::Key) -> QueryId {
// 1. 로컬 저장소에서 레코드 확인
// 2. 키에 가장 가까운 피어들에게 쿼리
// 3. 첫 번째로 찾은 레코드 반환
// 4. 캐싱 전략에 따라 추가 복제
}
```

3.3 프로바이더 레코드

```
#[derive(Clone, Debug)]
pub struct ProviderRecord {
```

```
/// 제공되는 값의 키
pub key: Key,
/// 값의 제공자
pub provider: PeerId,
/// 만료 시간
pub expires: Option<Instant>,
/// 제공자의 알려진 주소들
pub addresses: Vec<Multiaddr>,
}
```

프로바이더 시스템:

- 실제 콘텐츠 대신 콘텐츠 제공자 정보 저장
- 대용량 파일이나 스트림에 적합
- IPFS에서 널리 사용되는 패턴

4. 라우팅 테이블 관리

4.1 버킷 삽입 전략

```
#[derive(Copy, Clone, Debug, PartialEq, Eq)]
pub enum BucketInserts {
/// 연결 시 자동 삽입
OnConnected,
/// 수동 삽입만 허용
```

```
Manual,
}
```

4.2 피어 상태 관리

```
pub enum NodeStatus {
/// 현재 연결된 상태
Connected,
/// 연결이 끊어진 상태
Disconnected,
}
```

상태 전환:

- 새 연결 시: Disconnected → Connected
- 연결 종료 시: Connected → Disconnected
- 응답 없음 시: 버킷에서 제거 고려

4.3 대기 항목 처리

```
pub enum InsertResult<TKey> {
/// 성공적으로 삽입됨
Inserted,
/// 버킷이 가득 참
Full,
```

```
/// 대기 상태로 삽입됨
Pending { disconnected: TKey },
}
```

대기 메커니즘:

1. 버킷이 가득 찬 경우 대기 항목으로 등록
2. 기존 연결 끊어진 노드가 타임아웃되면 교체
3. 주기적으로 대기 항목들을 확인하여 적용

5. 쿼리 실행 엔진

5.1 쿼리 설정

```rust
#[derive(Debug, Clone)]
pub(crate) struct QueryConfig {
/// 단일 쿼리 타임아웃
pub(crate) timeout: Duration,
/// 복제 인수
pub(crate) replication_factor: NonZeroUsize,
/// 병렬성 수준
pub(crate) parallelism: NonZeroUsize,
/// 분리된 쿼리 경로 사용 여부
pub(crate) disjoint_query_paths: bool,
```

```
}
```

5.2 쿼리 통계

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct QueryStats {
requests: u32, // 총 요청 수
success: u32, // 성공한 요청 수
failure: u32, // 실패한 요청 수
start: Option<Instant>, // 시작 시간
end: Option<Instant>, // 종료 시간
}
```

5.3 반복 쿼리 알고리즘

```
1. 초기화:
- 목표 키 설정
- 로컬 라우팅 테이블에서 α개 가장 가까운 피어 선택
- 쿼리 상태 초기화

2. 반복:
- 현재 가장 가까운 α개 피어에게 요청 전송
- 응답 대기 (병렬 처리)
```

```
- 새로운 피어 정보로 후보 목록 업데이트
- 더 가까운 피어가 없으면 종료

3. 종료 조건:
- k개의 가장 가까운 피어 확보
- 더 이상 가까운 피어를 찾을 수 없음
- 타임아웃 발생
```

6. 보안 및 신뢰성

6.1 분리된 쿼리 경로 (Disjoint Query Paths)

```
// S/Kademlia 논문에 기반한 보안 강화
cfg.disjoint_query_paths(true);
```

특징:

- 악의적인 노드에 대한 복원력 향상
- 여러 독립적인 경로를 통한 쿼리
- 병렬성 수준만큼의 분리된 경로 사용

6.2 레코드 필터링

```
#[derive(Copy, Clone, Debug, PartialEq, Eq)]
pub enum StoreInserts {
```

```
/// 필터링 없이 즉시 저장
Unfiltered,
/// 이벤트를 통한 필터링
FilterBoth,
}
```

6.3 TTL 및 만료 관리

```
impl Record {
/// 주어진 시점에서 레코드가 만료되었는지 확인
pub fn is_expired(&self, now: Instant) -> bool {
self.expires.is_some_and(|t| now >= t)
}
}
```

7. 성능 최적화

7.1 캐싱 전략

```
#[derive(Debug, Clone)]
pub enum Caching {
/// 캐싱 비활성화
Disabled,
/// 최대 피어 수만큼 캐싱 활성화
```

```
Enabled { max_peers: u16 },
}
```

7.2 주기적 작업

```
/// 프로바이더 레코드 재게시 작업
add_provider_job: Option<AddProviderJob>,

/// 일반 레코드 재복제 및 재게시 작업
put_record_job: Option<PutRecordJob>,
```

주기적 유지보수:

- 레코드 재게시: 24시간마다
- 레코드 재복제: 1시간마다
- 프로바이더 재게시: 12시간마다
- 부트스트랩: 5분마다

7.3 메모리 효율성

```
// SmallVec 사용으로 작은 컬렉션 최적화
pending_rpcs: SmallVec<[(PeerId, HandlerIn); K_VALUE.get()]>,

// FnvHashMap 사용으로 해시 성능 향상
```

```
connected_peers: FnvHashSet<PeerId>,
```

8. 실제 사용 예제

8.1 기본 설정

```
use libp2p::kad;

// Kademlia 동작 생성
let store = kad::store::MemoryStore::new(local_peer_id);
let mut behaviour = kad::Behaviour::new(local_peer_id, store);

// 서버 모드 설정 (DHT 참여)
behaviour.set_mode(Some(kad::Mode::Server));
```

8.2 레코드 저장

```
// 키-값 쌍 저장
let record = kad::Record::new("my-key", b"my-value".to_vec());
let query_id = behaviour.put_record(record, kad::Quorum::One)?;
```

8.3 레코드 검색

```
// 키로 값 검색
```

```
let key = kad::RecordKey::new("my-key");

let query_id = behaviour.get_record(key);
```

8.4 프로바이더 등록

```
// 콘텐츠 제공자로 등록

let key = kad::RecordKey::new("content-key");

let query_id = behaviour.start_providing(key)?;
```

8.5 피어 검색

```
// 특정 키에 가장 가까운 피어들 찾기

let target_peer = PeerId::random();

let query_id = behaviour.get_closest_peers(target_peer);
```

9. 이벤트 처리

9.1 쿼리 결과 처리

```
match event {

SwarmEvent::Behaviour(kad::Event::OutboundQueryProgressed {

result: kad::QueryResult::GetRecord(Ok(record_result)),

..
```

```
}) => {
match record_result {
kad::GetRecordOk::FoundRecord(peer_record) => {
println!("레코드 발견: {:?}", peer_record.record);
}
kad::GetRecordOk::FinishedWithNoAdditionalRecord { .. } => {
println!("추가 레코드 없음");
}
}
}
// 기타 이벤트 처리...
}
```

9.2 라우팅 업데이트

```
SwarmEvent::Behaviour(kad::Event::RoutingUpdated {
peer,
is_new_peer,
addresses,
..
}) => {
if is_new_peer {
println!("새 피어 추가: {} at {:?}", peer, addresses);
```

```
}

}
```

10. 고급 기능

10.1 커스텀 레코드 저장소

```
use kad::store::RecordStore;


struct CustomStore {

// 커스텀 저장 로직

}


impl RecordStore for CustomStore {

// RecordStore 트레이트 구현

}
```

10.2 프로토콜 커스터마이징

```
// 커스텀 프로토콜 이름 사용

let protocol_name = StreamProtocol::new("/my-app/kad/1.0.0");

let config = kad::Config::new(protocol_name);
```

10.3 성능 튜닝

```
let mut config = kad::Config::new(protocol_name);

config

.set_query_timeout(Duration::from_secs(30))

.set_replication_factor(NonZeroUsize::new(10).unwrap())

.set_parallelism(NonZeroUsize::new(5).unwrap())

.set_max_packet_size(1024 * 1024); // 1MB
```

11. 모니터링 및 디버깅

11.1 쿼리 통계

```
// 쿼리 진행 상황 모니터링

for query in behaviour.iter_queries() {

let stats = query.stats();

println!("쿼리 {}: 요청 {}, 성공 {}, 실패 {}",

query.id(),

stats.num_requests(),

stats.num_successes(),

stats.num_failures());

}
```

11.2 라우팅 테이블 상태

```
// 라우팅 테이블 버킷 정보

for bucket in behaviour.kbuckets() {

let (min, max) = bucket.range();

println!("버킷 범위: {:?} - {:?}, 항목 수: {}",

min, max, bucket.num_entries());

}
```

결론

Rust libp2p의 Kademlia DHT 구현은 다음과 같은 특징을 가집니다:

1. **모듈식 설계**: 저장소, 쿼리 전략, 캐싱 정책을 독립적으로 구성 가능
2. **성능 최적화**: 비동기 처리, 메모리 효율적인 자료구조, 스마트 캐싱
3. **보안 강화**: 분리된 쿼리 경로, 레코드 필터링, TTL 관리
4. **확장성**: 대규모 네트워크를 위한 효율적인 라우팅 및 복제
5. **실용성**: IPFS 호환성, 다양한 사용 사례 지원

이러한 구현은 분산 애플리케이션에서 안정적이고 효율적인 DHT 서비스를 제공하며, P2P 네트워크의 핵심 인프라로 활용될 수 있습니다.